ANTIDOTE: Adaptive Continual Learning for Detecting Evolving Prompt Injection Attacks

Oluwadabira Omotoso
University of British Columbia
Kelowna, Canada
dabbyomotoso@gmail.com

***Abstract*—Large Language Models (LLMs) are increasingly deployed in real-world applications, making them attractive targets for prompt injection attacks, which OWASP currently ranks as the top security risk for LLM systems. Existing detection methods are fundamentally static, trained once on fixed datasets and unable to adapt as attack patterns evolve. This paper proposes ANTIDOTE, a continual learning framework that combines Elastic Weight Consolidation (EWC) and experience replay to enable a DeBERTa-v3-base classifier to adapt sequentially across evolving attack types without catastrophic forgetting. Evaluated across three sequential benchmarks including LLMailInject, HackAPrompt, and BIPIA, EWC combined with experience replay achieves an average F1 of 0.8408 and a backward transfer of -0.0584, substantially outperforming naive sequential fine-tuning and approaching the static joint upper bound. Few-shot zero-day evaluation further demonstrates that continual pretraining improves adaptation to unseen attack types by up to 15% over a from-scratch baseline. Our source code is publicly available at here. Video demonstration is publicly available at here.**

***Index Terms*—Prompt Injection, Continual Learning, EWC, Replay**

I. Introduction

LLMs are increasingly deployed across many fields, from general-purpose assistants to specialised applications and autonomous agent systems. As adoption grows, so does exposure to adversarial misuse. As of 2025, OWASP's top 10 for LLM Security identifies prompt injection as the highest security risk, occurring when a user's prompt alters the LLM's behaviour in unintended ways such as enabling unauthorised access or generating harmful content [1].

Prompt injection is not a static threat. Early direct injection attacks [2] operated through simple instruction overrides, giving way to indirect and RAG-based attacks [3] and more recently gradient-optimised adversarial payloads [4, 5, 6]. As surveyed by [7], this progression spans 26 distinct attack methods between 2022 and 2025, encompassing new vectors and optimisation strategies that no model trained at a fixed point in time could fully anticipate.

Despite this evolution, existing detection methods remain fundamentally static, trained once on fixed datasets with no mechanism for adaptation as attack patterns shift [8, 9]. This introduces catastrophic forgetting, where static detectors trained on earlier attack distributions lose discriminative power over emerging ones. Notably, while continual learning has been applied to similar cybersecurity problems such as IoT anomaly detection and vehicular intrusion detection [10], no prior work has framed prompt injection detection as a continual classification problem over evolving attack streams.

This paper addresses that gap directly. We propose a continual learning framework for prompt injection detection, built on a pre-trained transformer and trained sequentially across a stream of attack tasks that mirrors the real-world evolution of the threat landscape. Our key contributions are as follows:

- **A continual learning framework for prompt injection detection** combining Elastic Weight Consolidation (EWC) and experience replay.
- **A three-stream task formulation** structured as sequential classification tasks across three publicly available benchmarks.
- **Empirical evaluation** measuring backward transfer, forgetting, and per-task F1 scores, along with a few-shot zero-day simulation to evaluate how well the model generalises to previously unseen attack types.
- **Demonstration** that EWC combined with experience replay achieves strong performance comparable to static joint training, while outperforming naive sequential fine-tuning across all evaluation conditions.

II. Related Work

Prompt Injection attacks exploit a fundamental limitation in how transformer-based language models process input. This is because developer instructions and user data all share the same natural language format and they are all uniformly handled through self-attention mechanisms, so the model has no reliable way to distinguish trusted prompts from malicious ones. This ambiguity allows an attacker to steer the model toward unintended behaviour by embedding malicious instructions directly in the input. The various kinds of attacks keep evolving rapidly.

A. Prompt Injection Attacks

Early work by Perez and Ribeiro [2] showed that even simple, manually written inputs could override system prompts. This helped define the basic threat model for prompt injection. Later research developed more structured approaches. For example,Liu et al. [4] introduced HOUYI, a black-box probing system that combines context inference, payload generation, and feedback. It achieved an 86.1% success rate on commercial systems.

A major development was the idea of Indirect Prompt Injection [3]. In this setting, harmful instructions are hidden in external sources such as retrieved documents or web pages, rather than being directly written by the user. As LLMs began to act as autonomous agents that complete tasks over multiple steps, these hidden instructions could persist across the entire working context of the system. More recent work shows that attacks are no longer limited to text. Wang et al. [11] demonstrated that combined visual and textual changes can take control of multimodal agents. Similarly, Wang et al. [12] introduced attacks that use small, hard-to-detect pixel changes to influence graphical interfaces, without using any text at all.

Overall, this line of research shows that the threat has moved beyond simple chat interactions with attack success rates often exceeding 90% on systems without defences [7]. This motivates my work. Static detection methods, which are trained on fixed sets of known attacks, are not well suited to a threat that is constantly changing in both form and delivery methods.

B. Detection-Based Defences

The defence mechanisms that have been developed to mitigate prompt injection attacks could be classified into two, detection-based defences and prevention-based defences [13]. Detection-based defences work by screening inputs before they reach the target model, flagging anything that looks like an injection attempt. Rather than modifying the underlying LLM, these methods act as a separate filter sitting in front of it.

Several approaches have been proposed along these lines. The simplest and most common are classifier-based methods, where a smaller model is trained to distinguish benign inputs from malicious ones. Meta's PromptGuard-86M [14] and fine-tuned DeBERTa-based classifiers [9] fall into this category, learning to detect injections from semantic and syntactic patterns in the input text. A different angle is taken by Attention Tracker [8], which requires no training at all and instead monitors how the model distributes attention during inference to spot suspicious behaviour. Other methods like SmoothLLM [15] and Erase-and-Check [16] take a perturbation-based approach, either introducing random noise to expose instability caused by adversarial inputs, or systematically removing suspicious segments to verify whether the output changes. More recent systems such as InjecGuard [17] and UniGuardian [18] build on these ideas by combining multiple signals and refining detection thresholds to reduce false positives.

However, all of these methods share a fundamental limitation: they are static. Once trained, they cannot update themselves as new attack patterns emerge. As documented in [7], detection rates for novel or adaptive attacks can fall below 40%, which directly motivates the continual learning approach proposed in this paper.

C. Prevention-based Defences

Prevention-based defences take a fundamentally different approach to detection-based ones. Rather than flagging attacks as they appear, these methods attempt to make the system inherently difficult to manipulate, either through architectural restructuring, model alignment, or prompt engineering.

Several systems follow this approach. Methods like SecAlign [19] and StruQ [20] fine-tune LLMs directly on safety objectives, training them to reject malicious instructions by design or structuring inference to block injection propagation in agentic settings. Others like Spotlighting [21] and NeMo Guardrails [22] take a prompt engineering angle, labelling inputs by provenance or wrapping them with protective instructions to preserve system control. SELF-GUARD [23] trains models to detect and avoid unsafe responses, while more recent systems like CaMeL [24] and PIGuard [25] embed security principles directly into agent architectures.

While these approaches offer meaningful resilience, they are resource-intensive, typically requiring white-box access to the model or significant retraining overhead. Our work is explicitly complementary to this paradigm. Rather than replacing prevention-based defences, ANTIDOTE operates as a black-box screening layer that requires no access to the underlying LLM and no retraining of the target model, making it well-suited for integration alongside these strategies as part of a layered defence architecture.

D. Continual Learning

Continual learning studies how to train neural networks on a sequence of tasks without losing performance on earlier ones. The main challenge known as catastrophic forgetting, first identified by McCloskey and Cohen [26], where new training overwrites parameters that were important for previous tasks. This problem was later formalised for deep networks by Kirkpatrick et al. [27], who showed that sequential training on non-stationary data leads to rapid degradation in earlier task performance, especially as model size and task differences increase. A widely used solution is Elastic Weight Consolidation (EWC) [27], which uses the Fisher Information Matrix to estimate parameter importance and applies a regularisation penalty to limit changes to those parameters during new training. Another approach is experience replay, which stores samples from previous tasks and mixes them into later training batches, allowing the model to continue receiving gradient signal from past data while learning new tasks.

Continual learning can be studied in different settings. Despite extensive research, its application to security focused Natural Language Processing (NLP) tasks remains limited, and to our knowledge it has not yet been applied to prompt injection detection, which motivates the approach proposed in this paper.

Differences between our work and existing literature. Most prompt injection attack defence systems use a static approach. Our proposed method implements continual learning, specifically EWC and experiental replay to identify various prompt injection attacks. We propose that with this framework, we can handle the security of various prompt injection attacks.

III. Methodology

In this section, we describe the problem formulation, the base model used, existing baselines, the proposed method and

experimental setup, and the evaluation metrics.

A. Problem Formulation

We treat prompt injection detection as a binary text classification problem. Each training example is a single piece of text taken from our datasets (for example, an email subject or body, a jailbreak prompt, or a context segment with or without an attack). The classifier reads this text and predicts one of two labels:

- **benign (0):** normal content that does not contain a successful injection, or
- **injected (1):** content that contains or represents an injection attempt (for example, an attack string, a successful jailbreak prompt, or a context+attack pair).

In other words, the input to the model is just one text field from the dataset, and the output is a yes/no label $y \in \{0, 1\}$ indicating whether that text should be treated as injected or benign.

In the real world, new kinds of prompt injections appear over time, most attacks do not belong to one big dataset that a model was trained on. Instead, we see waves of attacks. Our goal is therefore to build a model that can be updated as new attacks show up, without losing what it already learned. This is where continual learning comes in and solves this problem. To simulate this real-world timeline of attacks. The data was split into three tasks. Task 1 uses LLMail-Inject [28], task 2 uses HackAPrompt [29], and task 3 uses BIPIA [30]. Rather than training once on a static mixture of all tasks, we train the same model sequentially on Task 1, then Task 2, then Task 3, and after each stage we evaluate it on all tasks seen so far. This continual learning setup allows us to study and mitigate catastrophic forgetting, where naive fine-tuning on new attacks can overwrite the model's ability to detect earlier ones.

In our experiments, we incorporate this continual learning setup with three successive task streams, each derived from a different real-world prompt injection dataset and capturing different kinds of attacks.

For Task 1, LLMail-Inject, this task is built from the LLMail-Inject [28] dataset of indirect email injection attacks. We use the *defense.undetected* flag as the main signal: when this flag indicates that the defence failed, we treat the example as injected and take the email subject line as the input text (after trimming leading and trailing white-space and requiring a minimum length). When the flag indicates that the defence succeeded, we treat the example as benign and take the email body as the input text (again trimmed and required to be long). We then balance the two classes by undersampling the larger one and, if necessary, cap the total number of examples. This results in a roughly balanced collection of email texts labelled as injected or benign.

For task 2, HackAPrompt, this task is built from the HackAPrompt [29] competition dataset of jailbreak prompts. Each example is a single jailbreak attempt, represented by the *user_input* field. We clean this text by removing leading and trailing white-space and removing very short prompts. The label in this task is derived from the *correct* field. Prompts that successfully bypass the target model are labelled as injected, while prompts that fail to bypass the model are labelled as benign. Just like task 1, we balance the two classes by undersampling the majority class and limit the total dataset size. This produces a diverse yet balanced set of jailbreak prompts labelled by success or failure.

For task 3, BIPIA, this task is constructed from Microsoft's BIPIA [30] benchmark for indirect prompt injection across code, email, and table domains. Using the *AutoPIABuilder* utilities provided in the BIPIA repository, we pair each clean context file (containing benign content) with its corresponding attack file. The builder inserts attack strings into the contexts at specified positions which produced "poisoned" contexts. These poisoned contexts are labelled as injected, while the aligned clean contexts, without any attack inserted, are labelled as benign. For every poisoned example we include one matched clean example, then shuffle and, if necessary, cap the dataset. This results in a balanced set of texts where the model must differentiate between clean external content and content that has been augmented with hidden attack instructions.

The task ordering reflects increasing detection difficulty. LLMail-Inject comes first as it establishes base detection capability against targeted, structured attacks. HackAPrompt follows second, introducing stylistic diversity through hundreds of human-crafted jailbreak strategies that force the model to generalise beyond the narrow LLMail distribution. BIPIA comes last as it presents the hardest challenge, with injections embedded inside legitimate external content across multiple domains rather than being the entire input.

B. Base Model

For our proposed method, we use DeBERTA-v3-base as the backbone model because it performs well on natural language inference and general text classification tasks, which fits our goal of deciding whether a prompt is benign or an injection attack. The base version was chosen as it offers a good balance between model capacity and computational efficiency, allowing it to capture subtle wording and long-range dependencies while remaining practical with limited GPU resources. On top of this backbone, we add a small classification layer that takes the final sentence representation and outputs a label (benign vs. attack). Both the backbone and this layer are trained together using cross-entropy loss, allowing us to make use of strong pre-trained representations while keeping the overall model simple and efficient for our prompt injection detection experiments with Elastic Weight Consolidation (EWC) and replay.

C. Baselines

Before introducing our continual learning method, we evaluate two simple, non-continual baselines that serves as reference points. Both baselines use the same backbone model and training budget as our proposed methods, but they do not apply any continual learning mechanism.

The first is static joint training. We use this as an upper bound i.e. we consider a "static joint" setting in which the

model is trained once on the union of all three tasks in an offline manner. We assume that there is full access to all data simultaneously and there is no sequential learning constraints. As a result, the model does not experience catastrophic forgetting, and its performance approximates what could be achieved if re-training on the entire dataset were always possible. This baseline therefore provides an upper-bound which we can compare more realistically with continual learning scenarios.

The second is Naive sequential fine-tuning. We use this as a lower bound. The model is first-trained on task T1 (LLMail), the further fine-tuned on T2 (HackAPrompt), and finally on T3 (BIPIA), without retaining data from previous tasks and without any explicit continual learning methods. This baseline mirrors a common practical workflow in which a single model is repeatedly fine-tuned as new data becomes available. However, it is known to suffer from catastrophic forgetting, as adapting to later tasks can degrade performance on earlier ones.

Together, these two baselines create a range that we aim to improve upon. Static joint provides an upper bound with no forgetting, while naive sequential fine-tuning represents a realistic but forgetful lower-bound. In our experiments, we report per-task F1 scores, average performance across tasks, and backward transfer (BWT) to quantify how our continual learning methods move the model closer to the static joint upper bound while mitigating the forgetting observed in the naive sequential baseline.

D. Continual Learning Methods

This section highlights the continual learning methods we tried and focuses on our proposed method, EWC + replay.

The first continual learning method we tried was Elastic Weight Consolidation (EWC). EWC follows a similar approach like naive sequential fine-tuning, but the major difference is that with EWC, after each task, it measures which weights were important for task using a Fisher matrix, and then it saves a copy of the important weights. When training on later tasks, we do not optimize only the current-task loss $(L_B(\theta))$, where $(L_B(\theta))$ is the usual cross-entropy loss on the current task (B); instead we add a quadratic penalty that discourages important parameters from drifting too far from their previous values:

$$L(\theta) = L_B(\theta) + \sum_{k \in past_tasks} \sum_i \lambda F_{k,i} (\theta_i - \theta^*_{k,i})^2$$

.

Here, (θ) are the current model parameters, $(\theta^{k,i})$ is the value of parameter (i) after training on past task (k), (Fk, i) is the (diagonal) Fisher information for parameter (i) on task (k), and (λ) is a scaling hyperparameter that controls how strongly we penalise changes to past-task parameters. The penalty is quadratic in the distance $((\theta_i - \theta^*_{k,i}))$, so parameters that move further away from their old values are penalised more, and parameters with higher Fisher values $(F_{k,i})$ are penalised more strongly because they were more important for those tasks.

We follow the known EWC implementation. That is, after each task, we compute the diagonal Fisher matrix for that task. The Fisher information tells us how much the model's prediction changes if we nudge each parameter. For example, we can deduce that the bigger Fisher value for a parameter suggests that the parameter was much important for that task so, changing it will hurt task performance. In our experiment, we focus only on diagonal Fisher matrix because the full Fisher matrix is large yielding a single importance score $(F_{k,i})$ per parameter. Overall, the diagonal Fisher helps us answer the question of "which weights should we protect?". After computing it, we store both the diagonal Fisher vector $(F_{k,i})$ and a snapshot of the corresponding optimal parameters $(\theta^*_{k,i})$ for each task (k).

The second continual learning method we tried was Experience Replay. When we fine-tune the model on each task, we maintain a small replay buffer of past examples and mix them into training batches for later tasks. In simple terms, we do not just train the model on the new tasks, instead, we keep a small "memory" of past examples and keep showing them to the model while it learns the new task, so it does not forget.

After completing each task (k), we randomly sample a subset of that task's training data and append it to a replay buffer with a fixed maximum capacity (5000 examples). When the buffer becomes full, we downsample it to keep roughly half of the stored examples, freeing space for newer samples while still retaining a diverse mixture of earlier tasks. During training on a later task (i), each optimisation step draws a batch (B_i) from the current task, and when the buffer is non-empty, a batch $(R_i)$from the replay buffer. The loss function is the sum of the two standard cross-entropy terms:

$$L_{\text{CE}}^{\text{current}}(\theta, B_i) + L_{\text{CE}}^{\text{replay}}(\theta, R_i),$$

where $(L_{\text{CE}}^{\text{current}}(\theta, B_i))$ is the supervised classification loss on current-task examples and $(L_{\text{CE}}^{\text{replay}}(\theta, R_i))$ is the loss on replayed examples from past tasks.

Our proposed method combined the discussed continual learning methods. We proposed EWC combined with Experience Replay as the ultimate adaptive framework. EWC protects the model structurally, while replay provides direct gradient signal from old data. We optimise a loss that adds both a replay term on past examples and an EWC penalty on Fisher-important parameters:

$$L_{\text{total}}(\theta) = L_{\text{CE}}^{\text{current}}(\theta, B_i) + L_{\text{CE}}^{\text{replay}}(\theta, R_i) + \lambda \sum_{k \in \mathcal{T}_{\text{past}}} \sum_i F_{k,i} \left(\theta_i - \theta^*_{k,i}\right)^2$$

$$\therefore L_{\text{total}}(\theta) = L_{\text{CE}}(\theta) + \lambda \cdot L_{\text{EWC}}(\theta)$$

This joint method lets the model both revisit previous attacks in input space and constrain key weights in parameter space, addressing catastrophic forgetting at different levels.

E. Evaluation Metrics

We use the following evaluation metrics to assess our models.

Per-task F1 score. For each task, we compute the F1 score on a held-out test set after each training stage. This tells us how

well the model distinguishes benign from prompt-injection examples on that specific task, and how its performance on each task evolves as new tasks are introduced.

Average accuracy across all tasks. After the model has been trained on all tasks in sequence, we compute the mean of the final per-task F1 scores, which we refer to as Average Accuracy. This provides a single summary value that reflects overall detection performance across all tasks at the end of continual training.

Backward transfer (BWT). To quantify forgetting, we compute the standard Backward Transfer (BWT) metric. For each task, we compare the F1 score immediately after the task is learned to the F1 score on the same task after training on all later tasks, and then average this change over all past tasks. Negative BWT indicates that performance on earlier tasks has degraded (catastrophic forgetting), while values closer to zero indicate better retention of previously learned knowledge.

Few-shot zero-day simulation. To evaluate performance novel attacks, we simulate a zero-day setting by holding out one prompt-injection dataset, then fine-tuning either a from-scratch model or a continual-learning model on only 5 to 10 labelled examples from the held-out dataset. We report the resulting F1 scores. Higher few-shot F1 for the continual learning model suggests that what it learned earlier can be reused, allowing it to adapt more easily and with less data to new types of attacks.

Ablation studies. We run ablation studies on the main continual-learning settings, focusing on the strength of Elastic Weight Consolidation (EWC) and the replay ratio. For each setting we report average accuracy and backwards transfer. These experiments show how our design choices change both final performance and forgetting, and they help us find configurations that balance learning new tasks with keeping performance on earlier ones.

IV. Results

This section presents the results of our experiments evaluating model performance and generalisation to novel attacks across a sequence of tasks. We first report overall performance in the main results table (Table I), followed by forgetting curves to analyse how performance changes across tasks. We then present ablation studies on the EWC λ penalty strength and replay buffer size to examine the impact of key design choices. Finally, we evaluate few-shot zero-day performance to assess adaptation to unseen attacks.

A. Main Results Table

Table I summarises the performance of all methods, reporting F1 on each task after the full training sequence (T_1, T_2, T_3), along with the average F1 and BWT. Static joint training achieves the best overall performance, with an average F1 score of 0.8889 across tasks. This is expected since it acts as an upper bound by training on all data jointly. In contrast, naive sequential fine-tuning shows strong catastrophic forgetting, with performance on T_1 dropping to 0.2998 and a negative BWT of -0.4438, indicating significant loss of earlier knowledge after learning new tasks. Both EWC only and replay only reduce this effect, increasing average F1 to 0.6488 and 0.8599, respectively, and improving BWT values to be closer to zero (-0.3468 and -0.0238). The combined EWC+replay method performs well overall, achieving an average F1 of 0.8408 with a small negative BWT of -0.0584, suggesting that combining regularisation and replay helps the model learn new tasks while largely retaining performance on earlier attack distributions.

TABLE I
MAIN RESULTS

Method	LLMail	HackAPrompt	BIPIA	Avg F1	BWT
Static Joint	0.9899	0.7938	0.8829	0.8889	N/A
Naive Sequential	0.2998	0.5437	0.8854	0.5763	-0.4438
EWC only	0.3594	0.7021	0.8847	0.6488	-0.3468
Replay only	0.9635	0.7365	0.8798	0.8599	-0.0238
EWC + Replay	0.9091	0.7318	0.8816	0.8408	-0.0584

Main Results on Three sequential prompt-injection detection tasks (T_1:LLMail, T_2:HackAPrompt, T_3:BIPIA). This table shows the performance across tasks, reporting F1 scores and backward transfer (BWT) for each method.

B. Forgetting Curves

The forgetting curves (Figure 1) show a clear separation between methods in how well they retain earlier tasks as training progresses. On LLMail (left panel), Naive Sequential drops sharply from near-perfect performance after T1 to roughly 0.2 after T2 and only recovers slightly by T3, indicating catastrophic forgetting. EWC only loses substantial T1 performance ending approximately around 0.3. Replay only and EWC + replay remain much more stable, both staying above 0.9 by the end of training.

On HackAPrompt (middle panel), all methods start similarly after T2, but Naive Sequential again declines the most by T3 (to about 0.5), whereas replay-based methods retain higher performance (around 0.7), with EWC Only in between. The BIPIA panel (right) reflects final task learning and shows similar end performance for replay-based methods. Overall, the figure indicates that replay is the best mechanism for remembering prior tasks, and combining replay with EWC leads to better retention, with more stable performance across tasks over time.

C. Ablation: EWC λ Penalty Strength

To study how sensitive EWC is to the regularisation strength, we vary the penalty weight λ over $\{100, 1000, 5000\}$ and report the resulting average F1 score and BWT in Figure 2. With a small value of $\lambda = 100$, the model does not sufficiently penalise changes to important parameters, resulting in both weak average F1 and strongly negative BWT, indicating substantial forgetting. When λ is increased to 1000, the model achieves its strongest average F1 across tasks; however, BWT worsens (becomes more negative), suggesting that moderate regularisation may inadvertently amplify forgetting on earlier tasks while optimizing for overall performance. Increasing λ further to 5000 reverses this trend: average F1 decreases

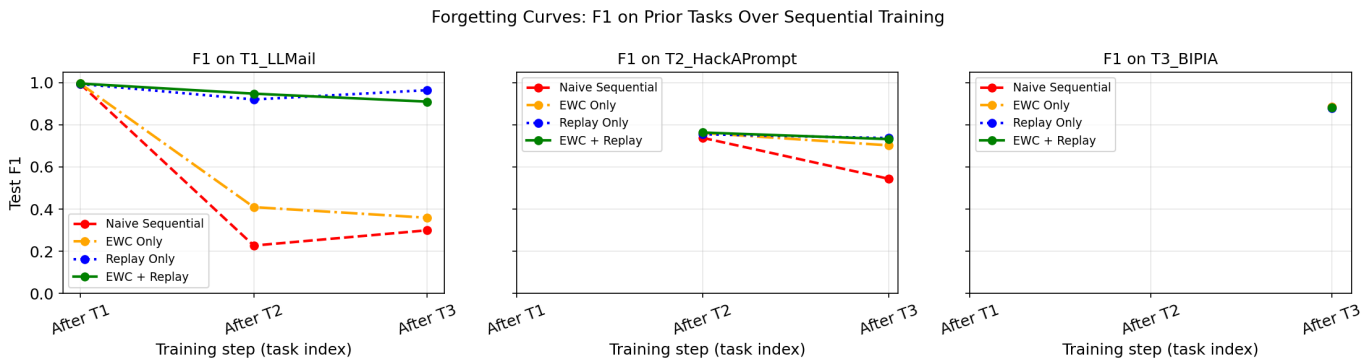

Fig. 1. Forgetting curves across sequential tasks (T1 to T3)

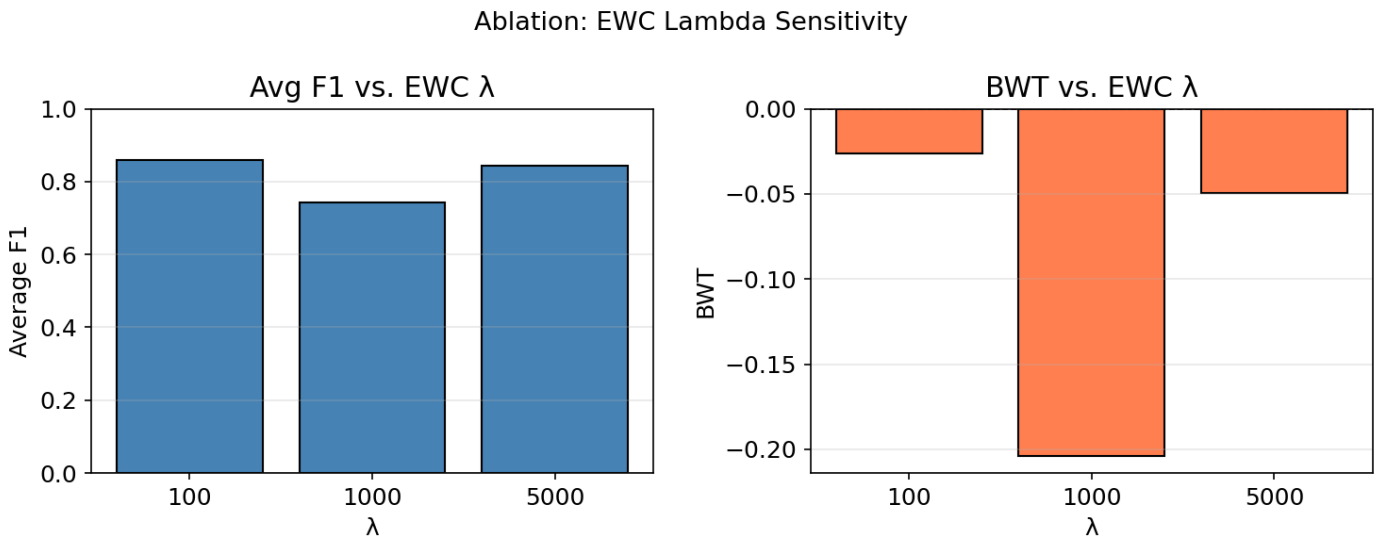

Fig. 2. EWC λ sensitivity analysis

slightly, but BWT improves considerably, indicating that stronger parameter constraints better preserve earlier knowledge at the expense of new-task learning. These results reveal a fundamental tension in the stability–plasticity dilemma [31]: no single λ optimises both metrics simultaneously, and the choice depends on whether the priority is maximising average task performance ($\lambda = 1000$) or minimising catastrophic forgetting ($\lambda = 5000$).

D. Ablation: Replay Buffer Size

To study how sensitive EWC+Replay is to replay buffer size, we vary the proportion of stored past examples over $5\%, 10\%, 20\%$ and report the resulting average F1 score and BWT in Figure 3. With a small replay ratio of 5%, the model achieves a reasonable average F1 but shows noticeable forgetting, reflected in a BWT of approximately -0.07. Increasing the replay ratio to 10% improves both metrics, with average F1 rising to about 0.83 and BWT becoming less negative at around -0.035, which indicates better retention of earlier tasks. Further increasing the replay ratio to 20% leads to additional but smaller gains, with average F1 reaching approximately 0.84 and BWT improving slightly to about -0.026. Overall, the results show a trade-off between memory usage and performance. A replay buffer size of 10% provides a strong balance between improved retention and efficiency, while the smaller gains at 20% suggest smaller gains when memory is limited.

E. Few-Shot Zero-Day Results

To evaluate how well the model adapts to unseen attack types, we test few-shot learning on a held-out zero-day task (T_3: BIPIA) using only 5 or 10 labelled examples. Figure 4 compares two initialisation strategies: training from scratch and initialising from a model pretrained with EWC+Replay on T_1 and T_2. In the 5-shot setting, the continual learning pretrained model achieves an F1 score of about 0.58, compared to 0.52 for the from-scratch baseline, giving an improvement of 0.06 points (around 12%). This difference becomes larger in the 10-shot setting, where the pretrained model reaches about 0.68 compared to 0.59 for the baseline, an improvement of 0.09 points (around 15%). These results show that pretraining with continual learning provides a better starting point for learning new attack types, especially when only a small number of examples are available. Overall, the findings

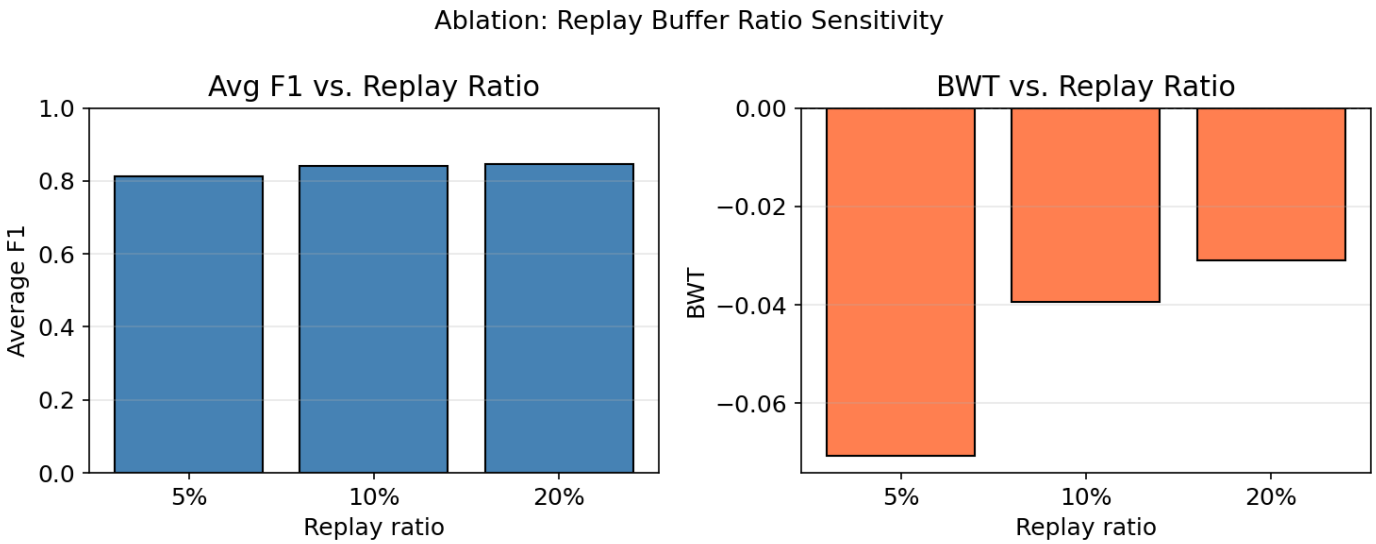

Fig. 3. Replay buffer size sensitivity analysis

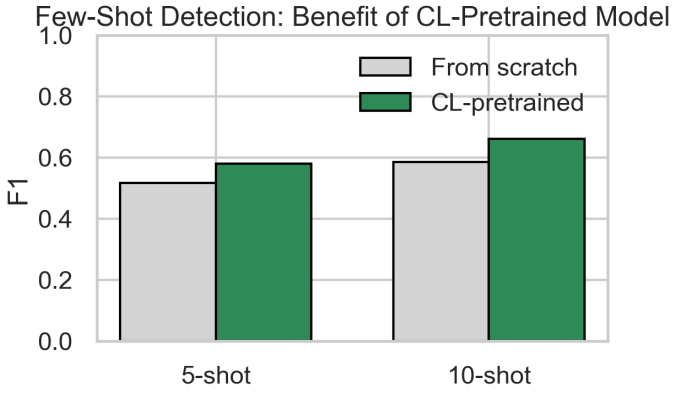

Fig. 4. Few-shot zero-day results

suggest that continual learning not only reduces forgetting on earlier tasks but also improves adaptation to previously unseen prompt-injection attacks.

Overall, while continual learning substantially improves retention and few-shot transfer, the results show that performance remains sensitive to regularisation and replay choices, indicating that the stability–plasticity trade-off is still an important limitation of the approach.

V. Discussion

The results confirm that framing prompt injection detection as a continual learning problem is both practical and necessary. EWC+Replay achieves an average F1 of 0.8408 and BWT of -0.0584, substantially outperforming naive sequential fine-tuning (0.5763, -0.4438) and approaching the static joint upper bound (0.8889). Experience replay is the primary driver of this retention, as replay only (BWT = -0.0238) outperforms EWC only (BWT = -0.3468) by a wide margin, suggesting that direct gradient signal from past examples is more effective than parameter regularisation alone. The few-shot zero-day results further show that continual pretraining improves adaptation to unseen attacks by roughly 12-15% over a from-scratch baseline, indicating that sequential training across diverse attack types encourages more general, transferable representations.

Despite these gains, important limitations remain. The lambda sensitivity analysis reveals a persistent stability-plasticity trade-off where no single regularisation strength simultaneously maximises average F1 and minimises forgetting, pointing to the need for adaptive regularisation strategies in the future. Additionally, all experiments use a binary/injected classification setup, which does not capture the full range of attack severity or intent. Extending this framework to support multi-class detection, larger backbone models, and longer attack streams would be valuable next steps towards a more robust and deployable continual detection system.

VI. Conclusion

This paper presented ANTIDOTE, a continual learning framework for prompt injection detection that addresses a fundamental limitation of existing static detection methods. By combining Elastic Weight Consolidation and experience replay on top of a DeBERTa-v3-base classifier, we demonstrated that a detection model can adapt sequentially across evolving attack distributions while largely preserving performance on previously learned tasks. Evaluated across three sequential benchmarks representing increasing attack complexity, EWC combined with experience replay achieved an average F1 of 0.8408 and a backward transfer of -0.0584, substantially outperforming naive sequential fine-tuning and approaching the static joint upper bound. Few-shot zero-day results further confirmed that continual pretraining produces more transferable representations, improving adaptation to unseen attack types by up to 15% over a from-scratch baseline.

These findings highlight that prompt injection detection is not a solved, static problem but an evolving classification challenge that benefits directly from continual learning. While limitations such as the stability-plasticity trade-off and the simplified task stream remain open challenges, this work

establishes a clear foundation for adaptive, deployment-ready detection systems. Future work should explore adaptive regularisation strategies, longer attack streams, and richer multi-class formulations to further close the gap between controlled evaluation and real-world deployment conditions.

Appendix

Reflection

When deciding on this project, I wanted to pick a topic I was genuinely interested in learning more about. At the time, prompt injection was a fairly new concept to me and I was also learning about intrusion detection systems. I thought it would be interesting to apply a deep learning concept to something within that space. While learning the course material, particularly neural networks and LSTMs, I kept thinking about how I could connect those concepts to what I had in mind. When we covered the lecture on weights and how to protect them and optimisation techniques, this sparked my interest in continual learning and pointed me in the direction of this project.

Through this project, I learned concepts stretching far beyond what was covered in class, which I consider a great personal achievement. I gained a much deeper understanding of the Fisher Information Matrix, the practical implementation of quadratic loss functions, and what hyperparameter tuning looks like in a real experimental setting. On the evaluation side, I also learned newer metrics like backward transfer, which gave me a principled way to quantify how much a model forgets over time.

The frequent feedback I received throughout the course contributed greatly to the continuous improvement of my project. When I first conceptualised this project, I expected my analysis to live mostly in a Jupyter notebook. However, while preparing for the project pitch, I realised that not everyone would be familiar with the problem, so I decided a simulation would be the best way to communicate my work. After the pitch, I received questions about whether I would include a model, how it compared to other frameworks, and whether it showed measurable improvement. Answering those questions gave me two clear goals: to quantify my results against static baselines so that others could understand how ANTIDOTE performs, and to include a checkpoint model in my demo to give users a live interactive experience of the system against one of those baselines. I also received feedback encouraging me to add visualisations to the report, which I acted on and felt made the results significantly more accessible and easier to interpret.

One limitation I encountered during the process was that I would have liked to include additional baselines such as PromptGuard or SecAlign. However, due to limited time and computing resources, I was constrained to presenting the static joint upper bound and naive sequential fine-tuning baselines. Despite this, I believe the chosen baselines still communicated the core findings effectively.

Overall, I really enjoyed working on this project. Although the experiments took a long time to run due to limited compute, the results were worth it. Going forward, I would like to continue this research, either by exploring how this framework could be applied to agentic workflows or by evaluating more adaptive regularisation strategies.

References

[1] OWASP Foundation, "Llm01: Prompt injection," 2024, https://genai.owasp.org/llmrisk/llm01-prompt-injection/, Last accessed on April 12, 2026.

[2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 2023, pp. 79–90.

[4] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, "Automatic and universal prompt injection attacks against large language models," *arXiv preprint arXiv:2403.04957*, 2024.

[5] D. Pasquini, M. Strohmeier, and C. Troncoso, "Neural exec: Learning (and learning from) execution triggers for prompt injection attacks," in *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*, 2024, pp. 89–100.

[6] C. Zhang, M. Jin, Q. Yu, C. Liu, H. Xue, and X. Jin, "Goal-guided generative prompt injection attack on large language models," in *2024 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2024, pp. 941–946.

[7] T. Geng, Z. Xu, Y. Qu, and W. E. Wong, "Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies," *Computers, Materials, & Continua*, vol. 87, no. 1, 2026.

[8] K.-H. Hung, C.-Y. Ko, A. Rawat, I.-H. Chung, W. H. Hsu, and P.-Y. Chen, "Attention tracker: Detecting prompt injection attacks in llms," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 2309–2322.

[9] Q. Lan, A. Kaul, and S. Jones, "Prompt injection detection in llm integrated applications," *International Journal of Network Dynamics and Intelligence*, vol. 4, no. 2, p. 100013, 2025.

[10] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE transactions on pattern analysis and machine intelligence*, vol. 46, no. 8, pp. 5362–5383, 2024.

[11] L. Wang, Z. Ying, T. Zhang, S. Liang, S. Hu, M. Zhang, A. Liu, and X. Liu, "Manipulating multimodal agents via cross-modal prompt injection," in *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025, pp. 10 955–10 964.

[12] X. Wang, J. Bloch, Z. Shao, Y. Hu, S. Zhou, and N. Z. Gong, "Webinject: Prompt injection attack to web agents," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 2010–2030.

[13] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847.

[14] Meta, "Promptguard-86m," 2024. [Online]. Available: https://huggingface.co/meta-llama/Prompt-Guard-86M

[15] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "Smoothllm: Defending large language models against jailbreaking attacks," *arXiv preprint arXiv:2310.03684*, 2023.

[16] A. Kumar, C. Agarwal, S. Srinivas, A. J. Li, S. Feizi, and H. Lakkaraju, "Certifying llm safety against adversarial prompting," *arXiv preprint arXiv:2309.02705*, 2023.

[17] H. Li and X. Liu, "Injecguard: Benchmarking and mitigating over-defense in prompt injection guardrail models," *arXiv preprint arXiv:2410.22770*, 2024.

[18] H. Lin, Y. Lao, T. Geng, T. Yu, and W. Zhao, "Uniguardian: A unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models," *arXiv preprint arXiv:2502.13141*, 2025.

[19] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "Secalign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 2833–2847.

[20] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "{StruQ}: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 2383–2400.

[21] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.

[22] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails," in *Proceedings of the 2023 conference on empirical methods in natural language processing: system demonstrations*, 2023, pp. 431–445.

[23] Z. Wang, F. Yang, L. Wang, P. Zhao, H. Wang, L. Chen, Q. Lin, and K.-F. Wong, "Self-guard: Empower the llm to safeguard itself," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 1648–1668.

[24] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[25] H. Li, X. Liu, N. Zhang, and C. Xiao, "Piguard: Prompt injection guardrail via mitigating overdefense for free," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 30 420–30 437.

[26] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of learning and motivation*. Elsevier, 1989, vol. 24, pp. 109–165.

[27] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[28] S. Abdelnabi, A. Fay, A. Salem, E. Zverev, K.-C. Liao, C.-H. Liu, C.-C. Kuo, J. Weigend, D. Manlangit, A. Apostolov *et al.*, "Llmail-inject: A dataset from a realistic adaptive prompt injection challenge," *arXiv preprint arXiv:2506.09956*, 2025.

[29] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, S. Anati, V. Tagliabue, A. Kost, C. Carnahan, and J. Boyd-Graber, "Ignore this title and hackaprompt: Exposing systemic vulnerabilities of llms through a global prompt hacking competition," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4945–4977.

[30] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2025, pp. 1809–1820.

[31] M. Mermillod, A. Bugaiska, and P. Bonin, "The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects," p. 504, 2013.